# SectorQ: Highly Scalable Pauli Propagation on HPC Systems with Window-Conserved Parity Sectors

ANONYMOUS AUTHOR(S), Submission under review, Anonymous

This paper presents *SectorQ*, a Pauli propagation system designed to compute the expectation values of quantum circuits with more than 100 qubits on GPU clusters. As tighter coefficient cutoffs increase the number of kept terms, the terms of a 127-qubit circuit exceed the memory of hundreds of GPUs, and the existing multi-GPU engine merges them across all GPUs after every gate. *SectorQ* introduces window-conserved parity sectors, which divide the terms by the parities that every gate of a window leaves unchanged and keep each new term in the same sector as its merge partner. With this design, *SectorQ* applies each window without exchanging terms between nodes, balances the sectors by choosing the parities at run time from a sample of the live terms, and holds the sectors in host memory beyond the capacity of the GPU memory. Our evaluation using seven circuits on up to 512 NVIDIA A100 GPUs shows that *SectorQ* reduces the simulation time of *cuPauliProp* on 16 GPUs by 4.4-9.7× and increases the largest finished problem on 512 GPUs by 140×.

## 1 Introduction

Quantum computers have advanced from demonstrations of quantum supremacy on sampling tasks to processors with more than 100 qubits that measure the expectation values of physical models [2, 10]. These expectation values are the output of quantum simulation, the application that motivates quantum computing in physics, chemistry, and materials science [7]. A 127-qubit processor, for example, measured the magnetization of a kicked Ising model at a scale beyond the exact classical methods [10]. This scale moves the comparison between quantum and classical computation from small benchmarks to the circuits of these applications. As a result, the range of expectation values that a classical computer can still produce has become a central question for the development of quantum computers.

Despite this progress, current processors remain noisy intermediate-scale quantum (NISQ) devices [14]. The noise of these devices biases every measured expectation value, and error mitigation recovers an estimate from repeated runs at amplified noise [10]. These estimates require an independent reference, and classical simulation on high-performance computing (HPC) systems provides this reference without noise. This reference verifies the hardware, calibrates the error mitigation, and tests quantum algorithms before they run on a processor. Thus, several groups simulated the 127-qubit experiment soon after its publication, with tensor networks and with Pauli propagation [3, 18].

Classical simulation of these circuits follows three approaches on HPC systems. State vector simulation stores all $2^n$ amplitudes of an $n$-qubit state, requiring 16 PiB for 50 qubits in double precision [8]. Tensor network simulation stores the state as a network of small tensors, and its cost grows with the entanglement of the circuit [18]. Pauli propagation tracks the observable, not the state, as a list of Pauli terms, and its cost follows the number of kept terms in place of the number of qubits [3, 17]. This cost extends Pauli propagation to circuits with more than 100 qubits, and NVIDIA and IBM now provide Pauli propagation in their quantum software stacks [13, 15]. Our work focuses on Pauli propagation, since its accuracy follows the resources of the system in place of the number of qubits.

The accuracy of Pauli propagation, however, requires far more terms than one GPU holds. A lower cutoff on the coefficients keeps more terms, and the terms grow within each layer of gates before the truncation removes the small ones. For example, the 127-qubit kicked Ising circuit retains up to 159 million terms at window boundaries at cutoff $10^{-5}$ and 11.9 billion at $10^{-6}$, requiring about

Author's Contact Information: Anonymous Author(s), Submission under review, Anonymous.

Table 1. Comparison with existing Pauli propagation systems (MN: Multi-Node, GPU: GPU Execution, LP: Locality-Preserving Partition, RP: Run-Time Partition from the Gates and the Live Terms, HM: Host Memory beyond GPU Memory)

| System | Method | MN | GPU | LP | RP | HM |
|---|---|---|---|---|---|---|
| PauliPropagation.jl [17] | Pauli propagation, one node | | | | | |
| SPD [16] | Sparse Pauli dynamics | | ✓ | | | |
| ORQA [5] | Structured distribution map | ✓ | | | | |
| MonoProp [1] | Distributed Pauli propagation | ✓ | | | | |
| *cuPauliProp* [13] | Global merge after every gate | ✓ | ✓ | | | |
| paulistrings-rs [12] | Fixed linear partition | ✓ | ✓ | ✓ | | |
| ***SectorQ* (Ours)** | **Window-conserved parity sectors** | ✓ | ✓ | ✓ | ✓ | ✓ |

6.4 GB and 475 GB of term storage before working buffers. These counts place Pauli propagation on HPC systems, and recent systems run it on GPU clusters and on the Fugaku supercomputer with $2^{17}$ processes [5, 13]. However, the existing systems show limited performance and scalability as the number of terms grows.

Existing distributed GPU engines face two limits, and the first is the global step after every gate. Since a new term may merge with a term on another GPU, the distributed mode of *cuPauliProp* coordinates the merge across GPUs after every gate [13]. This step adds a fixed cost to every gate, from 5.9 ms on 4 GPUs to 29.8 ms on 16 GPUs in *cuPauliProp*. The second limit is the GPU memory. Between two truncations, every rotation branches the terms, and the list grows to several times its size at the truncation. The merge of these terms also requires working space on the same GPU. As a result, *cuPauliProp* holds about 100 million kept terms on a GPU with 36 GB, and paulistrings-rs keeps its device partitions on the GPUs by the same rule [12]. A deferred merge reduces the global steps, but the unmerged copies of equal strings fill this memory faster. To summarize, these distributed GPU designs bind gate execution to global coordination and operator capacity to device storage, and our work aims to remove both constraints.

Many previous studies, as shown in Table 1, have optimized Pauli propagation from different algorithm and architecture perspectives. Single-node simulators reduce the cost of each gate with sparse data structures, symmetries of the circuit, and GPU kernels [3, 16, 17]. ORQA uses a structured distribution map to limit communication partners as it advances through generators [5], and MonoProp also supports distributed CPU propagation [1]. Distributed GPU engines keep the strings of each gate across all GPUs and merge them in a global step [13]. Linear partitions assign each string by parities of its bits, and these parities remain fixed for the whole run [12]. Theory on the commutator graph of a circuit also shows that the generators of the gates divide the Pauli strings into components that never mix [19].

*SectorQ* distinguishes itself from previous studies by recomputing the partition of the terms for every window of gates during the run. Previous systems merge the terms globally after every gate or fix the partition before the run, and these choices bind the exchange to the gates and the balance to rows chosen in advance. In contrast, *SectorQ* derives the parities that each window conserves from the generators of its gates, chooses among them by the balance of a sample of the live terms, and assigns each sector to one node for the whole window. Each GPU loads one sector at a time from the host memory of its node, and the terms move between nodes only at window boundaries. This design removes the global step after every gate and the bound of the GPU memory, and enables scalable performance across circuits, problem sizes, and GPU counts.

In this paper, we present *SectorQ*, a Pauli propagation system for GPU clusters that partitions the terms by the parities that a window of gates conserves. A rotation gate maps each term to itself and to its product with the generator of the gate, and every parity that the generator leaves unchanged assigns the same value to both terms. Specifically, *SectorQ* introduces 1) window-conserved parity sectors that the system recomputes for every window from the generators of its gates, 2) a run-time choice of the parities and the number of sectors from a sample of the live terms while the GPUs process the current window, and 3) a host-memory sector store from which each GPU loads one sector at a time. With these sectors, the terms move between nodes only at window boundaries, and the number of terms that *SectorQ* holds follows the host memory of all nodes. Our evaluation on up to 512 NVIDIA A100 GPUs of the Perlmutter supercomputer shows that *SectorQ* reduces the simulation time by 4.4-9.7× on 16 GPUs and increases the largest finished problem by 140× on 512 GPUs compared with *cuPauliProp*, the existing multi-GPU engine. We open-source *SectorQ* at: https://anonymous.4open.science/r/SectorQ

## 2 Background

### 2.1 Pauli Propagation beyond 100 Qubits

Classical simulation of a quantum circuit stores either the state or the observable. State vector simulation stores all $2^n$ amplitudes of the state, requiring 16 PiB for 50 qubits in double precision [8]. Tensor network simulation compresses the state into small tensors, and its cost grows with the entanglement of the circuit [18]. Pauli propagation tracks the observable, not the state, as a list of Pauli terms in the Heisenberg picture and computes its expectation value for circuits with more than 100 qubits [3, 17]. This list sets the cost of the simulation, and our work focuses on Pauli propagation, since its cost follows the number of kept terms in place of the number of qubits.

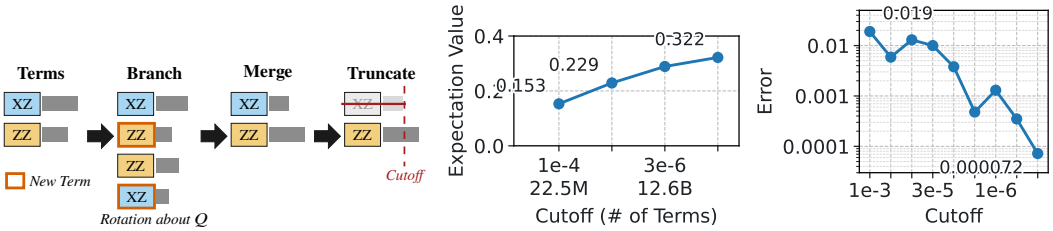
**Branch, Merge, and Truncate.** Each term of the list consists of a coefficient and a Pauli string, and the string maps to a bit vector with an x bit and a z bit for every qubit. With this bit vector, the product of two strings becomes the XOR of their bit vectors up to a phase. A rotation gate with generator $Q$ keeps every term that commutes with $Q$ and branches every anticommuting term $P$ into $P$ with $\cos\theta$ and $P \oplus Q$ with $\sin\theta$. These branches can double the number of terms at every rotation. The merge then sums the coefficients of the terms with equal strings, and the truncation removes the terms with a coefficient magnitude below the cutoff. Clifford gates map each string to one string and keep the number of terms, and noise channels scale the coefficients (Figure 1).

**Term Growth with Accuracy.** The cutoff of the truncation sets the accuracy and the number of terms together. On a 30-qubit circuit with an exact answer, the error decreases from 0.019 at cutoff $10^{-3}$ to 0.00007 at cutoff $10^{-7}$. On the 127-qubit kicked Ising circuit, the expectation value moves from 0.153 at cutoff $10^{-4}$ to 0.229 at $10^{-5}$ and 0.322 at $10^{-6}$. The largest boundary count rises to 159 million terms at cutoff $10^{-5}$ and 11.9 billion at $10^{-6}$. The latter occupies about 475 GB at 40 bytes per term before execution buffers. Within a window, we also sum the maximum term count reached by each sector execution. The largest of these sums reaches 1.4 billion and 100 billion at the two cutoffs, respectively. This counter captures intermediate expansion across sector executions, not a simultaneous memory footprint.

**Merge before Truncation.** The truncation must see the merged coefficient of each string. Equal strings that arise from different parents carry parts of one coefficient, and each part can lie below the cutoff although their sum exceeds it. A truncation of these unmerged parts removes this sum and changes the answer. For example, the 127-qubit kicked Ising circuit at cutoff $10^{-4}$ yields 0.194-0.283 when the truncation sees unmerged parts, whereas the merged run yields 0.1531. The merge partner of a new term depends on the gate that creates it and not on the location of its parent.

Table 2. Distribution of state vector simulation and Pauli propagation

| | **State Vector** | **Pauli Propagation** |
|---|---|---|
| Data | $2^n$ amplitudes of fixed size | Terms that grow and shrink at every gate |
| Location of the data | Fixed amplitude indices | Live strings and counts emerge at run time |
| Work of a gate | Updates fixed positions | Creates, merges, and removes strings |
| Communication | Gates on the qubits of the index | Every gate that may create a remote string |
| Limit | Memory of $2^n$ amplitudes | Number of kept terms |




(a) Operations of one gate (b) Kicked Ising value (c) 30-qubit TFIM error

Fig. 1. Operations and accuracy of Pauli propagation

An arbitrary distribution of the parents does not guarantee that their merge partners are local. A locality-preserving partition must account for the generators that create these partners (Table 2).

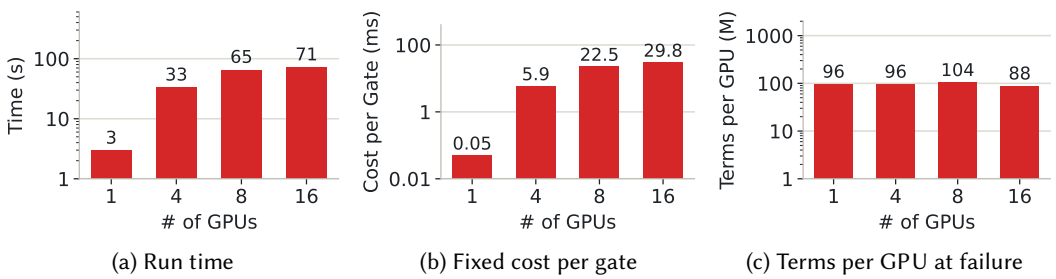
These two bottlenecks intensify at scale. The terms of an accurate run must spread over the GPUs of many nodes, whereas every equal string must meet its merge partners before each truncation. Existing systems run Pauli propagation on one device [3, 17], merge the terms across all GPUs after every gate [13], or fix the partition of the strings before the run [12]. A partition that follows both the gates and the live terms across nodes is not available in these systems.

## 2.2 Challenges in Distributed Pauli Propagation

Existing distributed systems merge the terms across all GPUs after every gate or partition the strings by rules fixed before the run [5, 12, 13]. We measure the first design with *cuPauliProp* v0.5.0 on the 127-qubit kicked Ising circuit and NVIDIA A100 GPUs, and the measurement exposes two challenges, a global step after every gate and a capacity bound by the GPU memory.

**Global Step after Every Gate.** *cuPauliProp* merges the terms of every gate in a global step, since a new term may merge with a term on any GPU. Each MPI rank applies the gate to its terms and calls allgather 22 times to merge and redistribute the terms with all other ranks, and 57% of the gates have no term to exchange. This global step adds a fixed cost of 5.9, 22.5, and 29.8 ms per gate on 4, 8, and 16 GPUs, against 0.05-0.8 ms on one GPU (Figure 2). As a result, a run with 22.5 million terms takes 2.9-3.2 s on one GPU and 33-48, 65, and 71 s on 4, 8, and 16 GPUs. The GPUs compute during 9% of this run, and the terms remain evenly spread at 1.001 times the mean on the largest rank. Thus, the cost of the global step, not an imbalance of the terms, slows the run as GPUs are added.

**Capacity Bound by the GPU Memory.** *cuPauliProp* also keeps all terms of every gate in GPU memory. Within a group of consecutive gates, the number of terms rises to 2.9-36 times its value at the start of the group in the seven circuits, and the merge requires working space next to these terms. As a result, *cuPauliProp* runs out of memory at 88-104 million terms per rank on 1 to 16




(a) Run time (b) Fixed cost per gate (c) Terms per GPU at failure

Fig. 2. Distributed mode of *cuPauliProp*

GPUs, and these kept terms occupy about 4 GB of the 36 GB that it uses on each GPU. A deferred merge reduces the global steps, but its unmerged copies exceed the memory of 16 GPUs on two of the seven circuits. A run without the global merge does not lift this bound and returns a wrong answer, 0.1675 on 4 GPUs and 0.189 on 16 GPUs against 0.1531, since its truncation discards parts of equal strings.

### 2.3 Opportunity in Parities Conserved by Windows

The two challenges require a partition that keeps every merge partner on the GPU of its parent without a global step. We examine where this partition exists with the generators of the seven circuits and with the terms of the kicked Ising circuit.

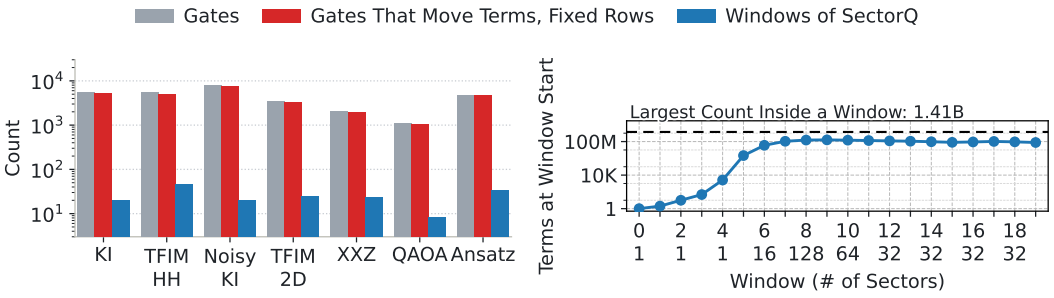
**Parities Conserved by Windows of Gates.** A rotation with generator $Q$ changes a term $P$ into $P \oplus Q$, and every parity $r$ with $r \cdot Q = 0$ assigns the same value to both terms. A group of consecutive gates, a window, conserves every parity orthogonal to all of its generators, and these parities divide the terms into sectors that keep the merge partners together. We count the windows in the seven circuits under the rule that a window grows while at least $n$ conserved parities remain (Table 3). The circuits with 1,100-7,960 gates form 8-45 windows, and a partition by window parities moves terms only at the window boundaries, 89-398 times fewer than the gates. In contrast, the whole circuit conserves at most two parities, and every term keeps the value of the observable on these parities. Thus, no parity divides the terms for the whole run, and the partition must change between windows.

**Fixed Partitions Move Terms.** Existing partitions fix their parities for the whole run and apply every gate to the terms [12]. Under these partitions, every gate that changes a fixed parity moves terms between processes. With 16 parts, rows over all bits move terms at 93-96% of the gates of the seven circuits, and rows over only the x bits or only the z bits move terms at 31-95% of the gates. On the kicked Ising circuit, random rows change the owner of 94% of the terms at a layer of two-qubit gates, and rows over lattice regions reduce this share to 68-89% with the largest of 16 parts at 1.65-3.37 times the mean. The library that implements these partitions keeps its rows for the whole run, since a changed row would move the owner of every term, and it lists the automatic choice of rows as an open problem [12].

**Partitions that Follow the Run.** The parities of each window follow its gates, and the balance of the sectors follows the live terms. On the kicked Ising circuit at cutoff $10^{-5}$, the terms at the start of a window grow from 1 to 159 million over the run, and the first windows hold too few terms to divide (Figure 3). The terms within a window then rise to 2.9-36 times this start in the seven circuits, and the sectors of a balanced partition differ from window to window. As a result, only the live terms of the run determine the number of sectors and the parities that balance them. Thus,

Table 3. Windows and fixed partitions in seven circuits (16 parts, rows fixed without a Clifford frame)

| Circuit | Gates | Windows | Gates per Window | Moving Gates (Random Rows) | Moving Gates (z Rows) |
|---|---|---|---|---|---|
| Kicked Ising | 5,420 | 20 | 271 | 94% | 50% |
| TFIM heavy-hex | 5,420 | 45 | 120 | 93% | 50% |
| Noisy kicked Ising | 7,960 | 20 | 398 | 95% | 50% |
| TFIM 11×11 | 3,410 | 24 | 142 | 94% | 60% |
| XXZ 8×8 | 2,048 | 23 | 89 | 94% | 62% |
| QAOA | 1,100 | 8 | 138 | 94% | 60% |
| Ansatz | 4,784 | 34 | 141 | 96% | 95% |




(a) Gates and windows

(b) Terms at window starts

Fig. 3. Windows of gates in seven circuits

Table 4. Requirements of distributed Pauli propagation

| # | Requirement | Evidence |
|---|---|---|
| R1 | Merge every equal string before each truncation | Unmerged parts change the answer |
| R2 | No coordination among GPUs during the gates | The fixed cost per gate grows with the GPUs |
| R3 | No accumulation of unmerged copies | A deferred merge exceeds the GPU memory |
| R4 | A partition recomputed during the run | Fixed rows move terms at 31-96% of the gates |
| R5 | A capacity beyond the GPU memory | Kept terms fill 4 of 36 GB on one GPU |
| R6 | No loss on small problems | 0.42 s on one GPU becomes 32-47 s on four |

distributed Pauli propagation requires a partition that the system recomputes for every window from its gates and from the live terms during the run.

These observations define six requirements for distributed Pauli propagation, listed in Table 4. A partition that keeps every merge partner in the part of its parent satisfies the first three requirements together.

## 3 SectorQ Design

*SectorQ* addresses the requirements in Table 4 with three mechanisms. Window-conserved parity sectors keep merge partners together throughout a group of gates. Runtime re-sectoring selects the next partition from the gates and the live terms during execution. Host-memory sector execution separates the stored expansion from the working set of each GPU and redistributes terms at window boundaries.

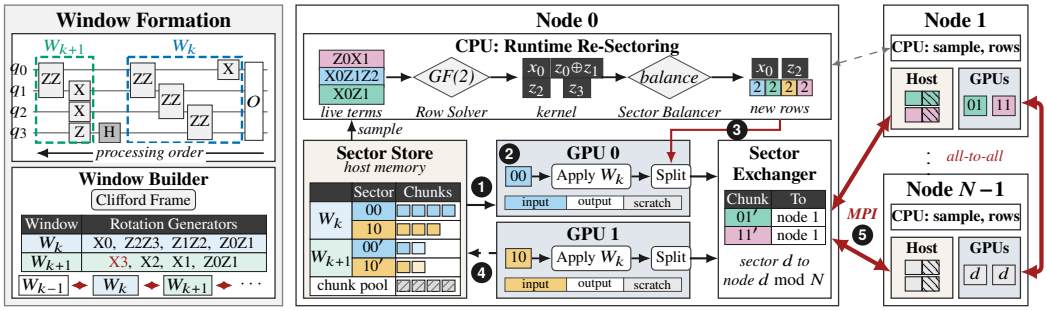



Fig. 4. Overall architecture of *SectorQ*

### 3.1 Overall Architecture

Figure 4 shows the circuit processing and the execution loop on each node. The CPU constructs windows and selects their rows, host memory holds the sectors, and each GPU processes one sector at a time.

**Window Formation.** The *Window Builder* first expresses the circuit in the processing order of the observable. A Clifford frame absorbs eligible Clifford gates and transforms the generators of subsequent rotations, preserving the action on the observable. For example, a Hadamard maps the generator $Z_3$ to $X_3$ in the illustrated circuit. For each window, the builder accumulates the span of the branching generators over GF(2) and stops before its dimension exceeds the number of qubits. This bound reserves a space of conserved rows from which the runtime can select a partition. The default boundary moves to the last scheduled truncation in the second half of the window when one is available, reducing the terms that require redistribution. Clifford operations that remain explicit transform generators into a common coordinate system for this calculation. The resulting window defines a sequence of operations and a space of valid partitions, with the particular rows selected during execution.

**Window Loop.** Each node follows the numbered steps in Figure 4. At step (1), a GPU worker loads one sector from the *Sector Store* in host memory. At step (2), its *Sector Executor* applies the current window, including the merges and scheduled truncations. During these operations, *Runtime Re-Sectoring* uses the CPU to select the rows of the next window. At step (3), the worker uses these rows to partition its output on the GPU. Step (4) appends output chunks to the host lists of the next sectors. The *Sector Exchanger* transfers the remote lists at step (5), assigning sector $d$ to node $d$ mod $N$ among $N$ nodes. Each node then processes its received sectors with the same loop. The last window evaluates the observable against the input state and reduces the partial answers across nodes. Thus, the data path repeats by window and keeps the distributed merge outside the sequence of gates within a sector.

### 3.2 Window-Conserved Parity Sectors

The execution loop requires a partition that preserves merge locality through every operation of a window. Figure 5 illustrates this condition on four qubits and two parity rows.

**Rows Conserved by a Window.** A Pauli string has a binary representation $v = (x, z) \in \mathrm{GF}(2)^{2n}$, with its phase represented in the coefficient. For a row $r$, the parity is the binary inner product $r \cdot v$. A branching rotation with generator $q$ creates strings $v$ and $v \oplus q$. Their parities agree whenever $r \cdot q = 0$. The *Row Solver* computes the annihilator of the generator span with Gaussian elimination over GF(2). If that span has dimension $d$, its annihilator has dimension $2n - d$, at least $n$ under

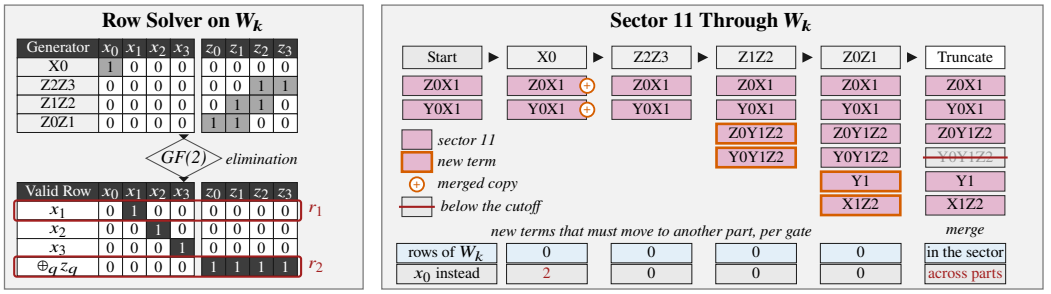



Fig. 5. Window-conserved parity sectors

the window rule. The selected rows form a matrix $R$, and the bit vector $Rv$ labels the sector of $v$. This binary linear algebra is related to symmetry detection for qubit tapering [4]. Here its role is to partition the operator during one window, rather than remove qubits using a symmetry of the entire problem.

**Merge Locality.** Every descendant of a term retains its sector label through the branching rotations of the window. This property follows by induction from $R(v \oplus q) = Rv$ for each generator in the window. A coefficient change from a Pauli noise channel also preserves the string and its label. For an explicit Clifford operation, the row coordinates transform with the strings to preserve the same label. Equal strings have equal labels, and all contributions to a string therefore belong to one sector before truncation. Consequently, a merge and coefficient truncation within each complete sector produce the same operator as a global merge and truncation at the same operations, apart from floating-point summation order. In the example, the rows $x_1$ and $z_0 \oplus z_1 \oplus z_2 \oplus z_3$ label both $Z_0X_1$ and $Y_0X_1$ as sector 11. Their descendants remain in this sector through $W_k$, including terms that merge or disappear at a truncation.

**Exchange Only at Window Boundaries.** The next window can contain generators that violate the current rows. For example, $X_1$ in $W_{k+1}$ changes the parity $x_1$ used by $W_k$. The runtime therefore computes a new matrix for $W_{k+1}$ and regroups the output of $W_k$ before the next window begins. This regrouping collects all fragments of each next sector on its owner, where equal strings meet before their next truncation. The construction differs from a fixed linear partition [12], whose rows can require a remote transfer at a gate that changes a partition label. It also differs from a partition based only on symmetries of the full circuit, since these symmetries need not distinguish the live terms. The window rule requires conservation only over the operations assigned to the current sector execution. Thus, the system obtains a new space of valid partitions at each boundary without changing the prescribed truncation schedule.

### 3.3 Runtime Re-Sectoring

Conservation determines which rows are valid, but the live terms determine their load balance. Figure 6 shows how the runtime selects the next rows while the GPUs process the current window.

**Sampling the Live Terms.** At the start of a window, each node samples strings from its host sectors with a regular stride determined by the global term count. The configured sample target is 200,000 terms across all nodes. The CPU applies the current window's bit-changing Clifford operations to these strings before the next partition is scored. This estimate omits branching and truncation, avoiding a second propagation of the sampled operator. The sample therefore approximates the coordinate distribution at the next boundary rather than predicting its exact population. An MPI gather collects the samples on rank zero, where the row solver and the *Sector Balancer* make a

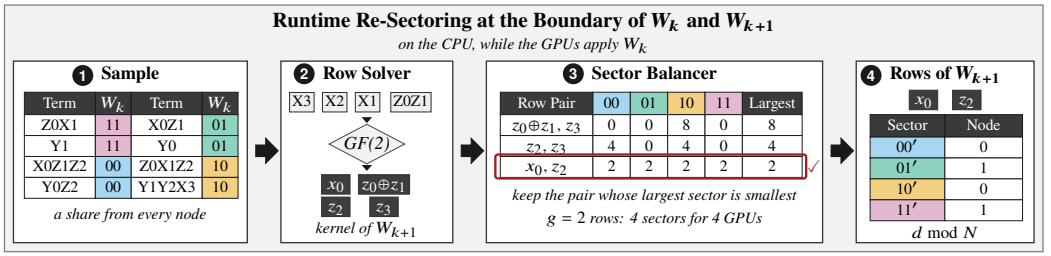



Fig. 6. Runtime re-sectoring at a window boundary

common decision for every node. A broadcast distributes the selected rows and window description. All workers consequently use the same sector labels, even though their local input sizes differ. The sample affects performance decisions only, since every selected row must satisfy the generator constraints independently of the sampled terms.

**Balanced Row Selection.** The sector balancer generates eight candidate row sets from random combinations of the annihilator basis. For each candidate, it counts sampled strings by sector and selects the set with the smallest largest count. The illustrated pairs produce largest counts of eight, four, and two, and the selected pair $(x_0, z_2)$ divides the eight strings equally. The number of rows follows the estimated output size, the target terms per sector, and the available GPUs. The estimate uses the previous window's observed growth with a bounded growth factor. The runtime requests one task per GPU, or two when the preceding window exceeds the configured duration threshold, provided the resulting sectors remain sufficiently large. Early windows with few terms retain a smaller partition. A configurable cap bounds the number of sectors at a boundary. These choices adapt task size and placement to the evolving operator without assuming that any valid row set guarantees balanced work.

**Overlap with the Running Window.** The host sample is captured before the workers consume their input sectors. After the workers start, the CPU main thread gathers the sample, computes the next window, scores the candidate rows, and broadcasts the result. Workers need those rows only when they partition their output for the next window. A worker that reaches this point before the rows are ready waits on a local condition variable. The implementation records this wait separately from the CPU selection time, allowing the evaluation to distinguish overlapped work from exposed delay. The process therefore has no separate global optimization phase between successive GPU executions. In contrast to a circuit schedule based solely on gates, the row selection responds to the distribution of terms present during the run. Correctness depends on the conservation constraints, whereas the value of the sample and candidate search depends on the resulting execution time and sector balance.

### 3.4 Host-Memory Sector Execution and Exchange

The selected sectors are units of execution rather than permanent GPU allocations. Figure 7 shows their storage, processing, and transfer across a window boundary.

**Chunked Sector Store.** The sector store represents each sector as a list of fixed-capacity host chunks, with 262,144 terms per chunk by default. Each term contains a binary string and a double-precision coefficient. A pool retains pinned chunks for reuse after GPU transfers or MPI sends, avoiding repeated registration for every sector. The same chunk representation supports input loading, output collection, and reception from another node. A sector can contain multiple chunks, and a node can hold more sectors than it has GPUs. The live expansion therefore occupies host

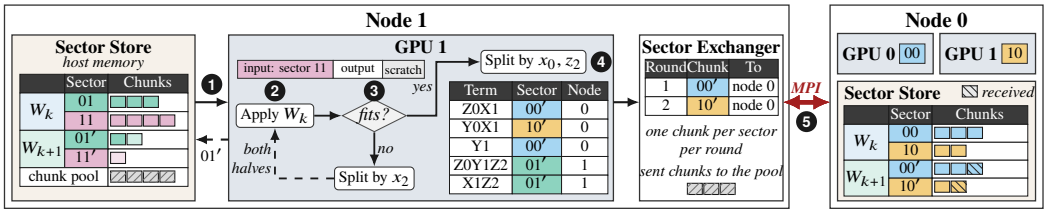



Fig. 7. Host-memory sector execution and boundary exchange. Pale sector colors denote filling chunks, black hatching received chunks, and gray hatching pooled chunks

memory while only the sectors under execution require GPU buffers. The total capacity depends on host storage, temporary output, and exchange buffers, rather than the sum of GPU buffers alone. Within a node, workers obtain sectors from a shared queue, initially ordered by decreasing size. This queue allows a GPU that finishes one sector to process another sector without a permanent binding between a sector and a device.

**Sector Execution on GPUs.** Each worker maintains input and output expansions together with the library workspace on its GPU. The worker loads a complete sector, applies its window, and performs the required merges before coefficient truncation. If the sector is too large to load with sufficient workspace, an additional valid row divides it in host memory. If growth exhausts the GPU buffers during execution, the worker partitions the current expansion using a further row conserved by the remaining operations. The resulting pieces return to the local queue with the operation position and any pending merge or truncation. Each piece resumes from that position, retaining the merge-locality argument within the finer partition. A split succeeds only if the available rows distinguish enough terms to fit the working set. The system therefore records both GPU and host splits, and the capacity claim includes the requirement that these finer sectors remain executable on a GPU.

**Boundary Exchange.** At the end of a window, a GPU partitions its surviving terms by the next matrix and copies them into host chunks. Chunks for local sectors remain on the node, and the sector exchanger sends the other chunks to their owners. The exchange first communicates counts and then transfers the chunks in rounds. For each sender and destination sector, a round transfers at most one chunk. After the outstanding requests complete, the sender returns transmitted chunks to the pool before the next round begins. The additional receive storage is therefore bounded by one round, and the request count depends on the active sender-sector pairs rather than all chunks in the expansion. At completion, the owner joins the received chunk lists into each complete next sector. A checkpoint can record this boundary state and its next-window description. Thus, the host representation supports both GPU execution and redistribution without a second full-size receive allocation.

### 3.5 Implementation

*SectorQ* implements the host runtime in C++ and the string partition operations in CUDA. Each MPI rank owns one node and creates one worker per GPU, with four GPUs per node in our experiments.

**Runtime and Library Operations.** Each worker uses *cuPauliProp* v0.5.0 on one GPU. The library operations cover rotations, Clifford gates, Pauli noise, merging, and truncation. The runtime supplies the sector contents and manages their placement, while the library supplies the arithmetic on each expansion. CUDA kernels evaluate the row parities and group output terms by their next sectors. The CPU handles binary elimination, sample scoring, the host chunk pool, and MPI exchange. Consecutive Pauli noise channels can share a library call up to the next truncation, preserving the

Table 5. Runtime parameters

| Parameter | Setting |
|---|---|
| Sample target across nodes | 200,000 terms |
| Candidate row sets | 8 |
| Target sector input | 4 million terms, 6 million for kicked Ising |
| Tasks per GPU | 1, or 2 after a window of at least 0.5 s |
| Default sector cap | 256 |
| Capacity experiment sector caps | 1,024 or 4,096 |
| Host chunk capacity | 262,144 terms, 65,536 for selected large runs |
| GPU memory budget | 90% of available device memory |
| Each expansion buffer | 20% of the GPU memory budget |
| Coefficient truncation | Every 10 operations by default |

scheduled coefficient decisions. On one GPU, an expansion below the target sector size can remain in device memory between windows. This path avoids a host transfer when no other worker could receive a sector. These execution choices complement the window partition but are not the source of its merge-locality property. Table 5 summarizes the default parameters and the larger sector caps used in the capacity experiments.

## 4 Evaluation

### 4.1 Evaluation Setup

We evaluate correctness, execution time, and capacity under explicitly matched inputs and truncation rules.

**Hardware Specification.** All experiments use GPU nodes of the NERSC Perlmutter system. Each node contains four NVIDIA A100 GPUs with 40 GB of device memory each, a 64-core CPU, and 256 GiB of host memory. The nodes communicate over the Slingshot network. GPU implementations use a memory budget of 90% of the available device memory, approximately 36 GB per GPU. *SectorQ* assigns one MPI rank to each node and one worker to each GPU. The distributed mode of *cuPauliProp* assigns one rank to each GPU. The scale experiments use up to 128 nodes and 512 GPUs, while the single-node experiments compare each implementation on the resources it supports. CPU implementations use the same nodes as the corresponding GPU configuration, with their process and thread counts stated below. The comparisons therefore identify both the number of nodes and the execution devices.

**Evaluation Scope.** The evaluation measures Heisenberg-picture propagation that returns an observable expectation value. A coefficient cutoff controls the retained operator size, and lower cutoffs usually require more terms. Unless stated otherwise, each setting is measured once with a 40-minute limit. The size and exact-reference experiments use a 20-minute limit, and the capacity ladders allow *SectorQ* up to two hours per run. A ladder stops after memory exhaustion or its time limit, and an unexecuted point is not a measured failure. Reported times cover the propagation region rather than batch queue or process-launch delays. Initialization performed lazily inside that region remains included. Term counts on the horizontal axes denote the largest count at a *SectorQ* window boundary under the stated truncation rule. These counts differ from the larger within-window population and from the final output size. Separate experiments measure repeated-run variability, preserving the single-run policy for the main sweeps.

**Circuits.** The seven circuits in Table 3 cover different interaction graphs, gate families, and observables. The principal workload is a 127-qubit heavy-hex kicked Ising circuit with 20 steps and

rotation angle $\pi/4$. A second heavy-hex circuit uses transverse-field Ising model (TFIM) rotations at general angles, and a noisy variant adds depolarizing channels with probability 0.005. The remaining circuits are a TFIM on an $11 \times 11$ lattice, an XXZ model on an $8 \times 8$ lattice, a 100-qubit QAOA circuit on a random regular graph, and a 100-qubit hardware-efficient ansatz. Their respective depths are 10 steps, 6 steps, 4 layers, and 16 layers. The observables include a local magnetization and the edge sum of QAOA. Additional TFIM grids and kicked Ising depths test whether the behavior extends beyond the main circuit sizes. All comparisons within a setting preserve its circuit, observable, and initial state.

**Baselines.** The primary baseline is *cuPauliProp* v0.5.0 with the original circuit and its default distributed mode. Our second baseline uses a Clifford-frame input and defers global deduplication until truncation, with additional merges where required to fit memory. We report the fastest successful deferred interval among those tested. The fixed-partition comparison uses paulistrings-rs with the original gates and matches its gate-level truncation rule. Other comparisons include the authors' SPD, MonoProp, PauliPropagation.jl, and ppvm implementations, Qiskit's Pauli propagation package, and our reproduction of ORQA. These external implementations, except paulistrings-rs, receive the exported Clifford-frame operation list. ORQA uses 64 ranks per node, MonoProp uses eight ranks with eight threads each, and the threaded single-node implementations use 64 threads. Qiskit uses one thread. MonoProp's truncation rule differs, and its timings are not treated as comparisons at identical numerical error. The ORQA results describe our reproduction rather than the authors' implementation.

### 4.2 Single-Node Performance

The single-node comparison establishes the arithmetic and memory behavior before inter-node exchange becomes necessary.

**Comparison with GPU Implementations.** Figure 8 compares implementations on one node as the kicked Ising cutoff decreases. On problems that fit one GPU, the measured ratio of the *cuPauliProp* single-GPU time to the *SectorQ* single-GPU time ranges from 0.6 to 1.5 across circuits. Both systems use the same library arithmetic, and the differences include circuit preprocessing and sector management. SPD is up to 2.9 times faster on one GPU in the tested settings, but its kicked Ising run exhausts memory at cutoff $10^{-5}$. With four GPUs, *SectorQ* completes kicked Ising at cutoff $3 \times 10^{-6}$ in 436 s, whereas the primary distributed baseline stops at a higher cutoff. At their shared cutoff $3 \times 10^{-5}$, the times are 5.25 and 44.3 s. The single-node gain therefore combines parallel execution with a capacity beyond the GPU-resident expansion of the baseline.

**Comparison with CPU Implementations.** The same-node comparisons also identify where CPU execution remains competitive. At the largest shared settings across circuits, *SectorQ* on four GPUs is 108-269 times faster than PauliPropagation.jl, 157-755 times faster than ppvm, and more than 1,800 times faster than the single-threaded Qiskit implementation. These ratios compare the available execution resources of each implementation rather than identical processor types. The ORQA reproduction is 18-71 times slower on the same node. MonoProp is faster on several small settings that finish within two seconds, including kicked Ising at cutoff $10^{-3}$. Its different truncation policy also requires a separate accuracy comparison. As the operator grows, *SectorQ* retains a larger executable problem range by keeping inactive sectors in host memory. Thus, the relevant benefit is the joint time and capacity behavior over the cutoff ladder, with the small-problem CPU results retained in the comparison.

### 4.3 Performance across Circuits on 16 GPUs

We next test whether the window construction benefits different circuit structures on the same resources.

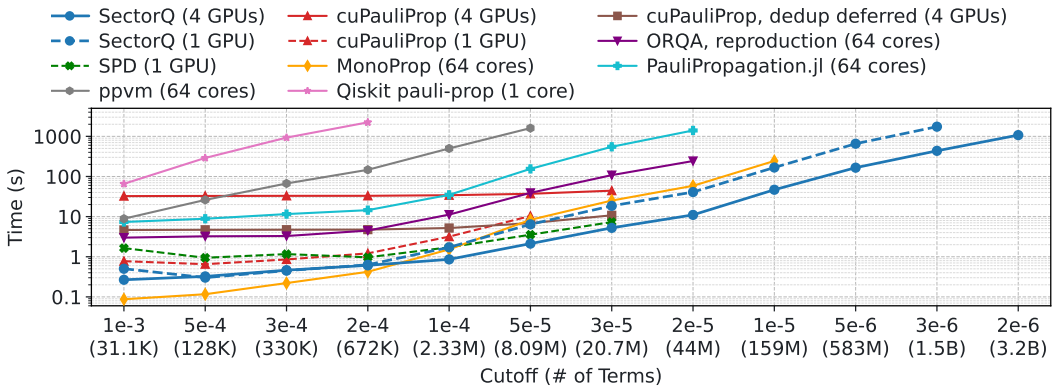



Fig. 8. Simulation time on one node (4 GPUs)

**Simulation Time.** Figure 9 compares seven circuits on the same four nodes and 16 GPUs. At the largest setting completed by the primary *cuPauliProp* baseline for each circuit, *SectorQ* is 4.4-9.7 times faster. For kicked Ising at cutoff $10^{-5}$, the original-circuit baseline takes 95.0 s and *SectorQ* takes 12.2 s. The comparison with deferred deduplication separates the benefit of fewer global merges from the benefit of window sectors. That baseline remains 2.2-4.6 times slower at the shared settings and exhausts memory in two circuits at the primary baseline's largest completed setting. *SectorQ* also completes lower cutoffs in all seven circuits. The gain appears both in circuits with Clifford gates and in TFIM and XXZ circuits whose input contains no removable Clifford gates. Consequently, the observed improvement extends beyond the reduced operation count of the Clifford frame.

**Time per Term.** Figure 10 divides propagation time by the largest boundary term count of the setting. This normalization measures time per retained term rather than the duration of a single gate application, since a term may participate in many operations. For settings above 100 million boundary terms, *SectorQ* takes 17-130 ns per term across the circuits. The curve becomes relatively flat as the arithmetic on large sectors dominates the fixed cost of library calls and window coordination. At cutoff $10^{-4}$, the original-circuit *cuPauliProp* baseline takes 908-39,000 ns per term. Its global operations retain substantial cost even when few terms survive. The difference decreases as a baseline run contains more useful arithmetic, consistent with the smaller speedups at its largest successful settings. The curves therefore connect the time comparisons to problem size without assuming identical work per term across different circuits.

**Agreement of Answers.** The completed runs with a common truncation rule agree across implementations and GPU counts. Across 68 circuit-cutoff settings with at least two successful runs, the largest absolute difference in the expectation value is $4.9 \times 10^{-11}$. The comparison includes the single-GPU reference, distributed *cuPauliProp*, and external implementations with the same coefficient decisions. The largest absolute difference occurs for QAOA, whose observable is an edge sum rather than one local Pauli expectation. *SectorQ* also preserves the printed answer when the number of sectors, GPU count, or window rule changes. These checks test the numerical consequence of merge locality through actual partition and exchange operations. They do not establish that a chosen cutoff approximates the exact quantum expectation sufficiently well. Section 4.9 separately compares a smaller circuit with an exact state-vector reference and reports the cutoff dependence of the larger kicked Ising workload.

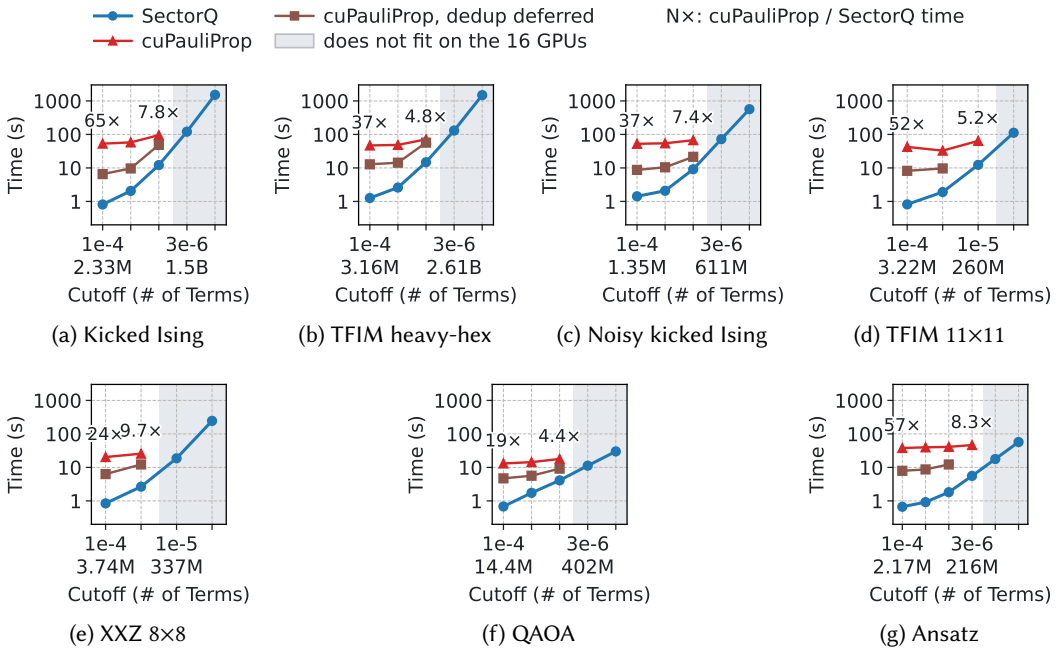



Fig. 9. Simulation time of seven circuits on 16 GPUs

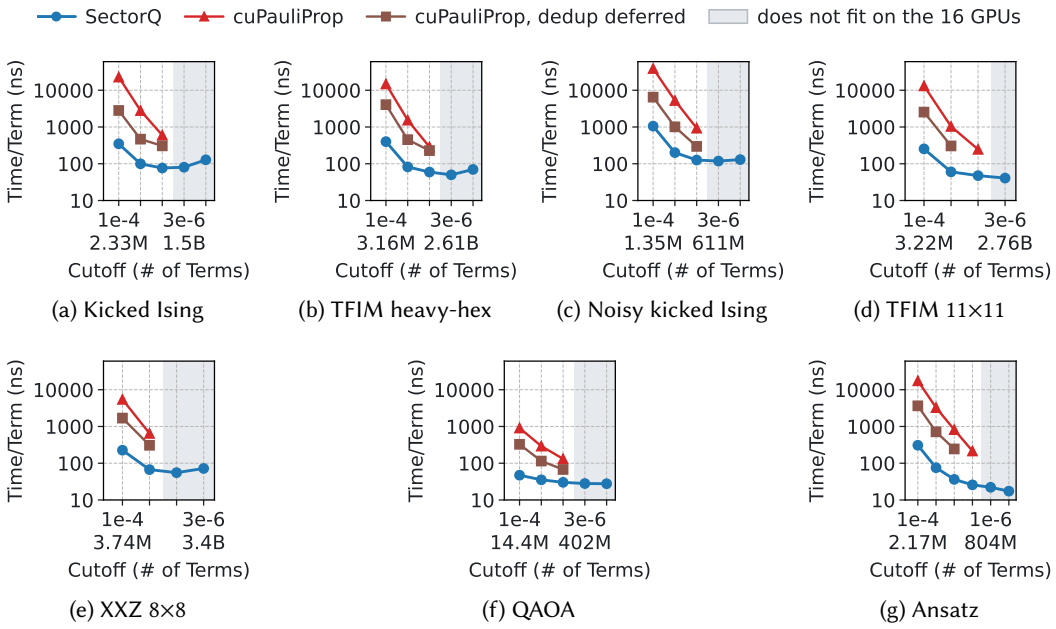



Fig. 10. Time per term of seven circuits on 16 GPUs

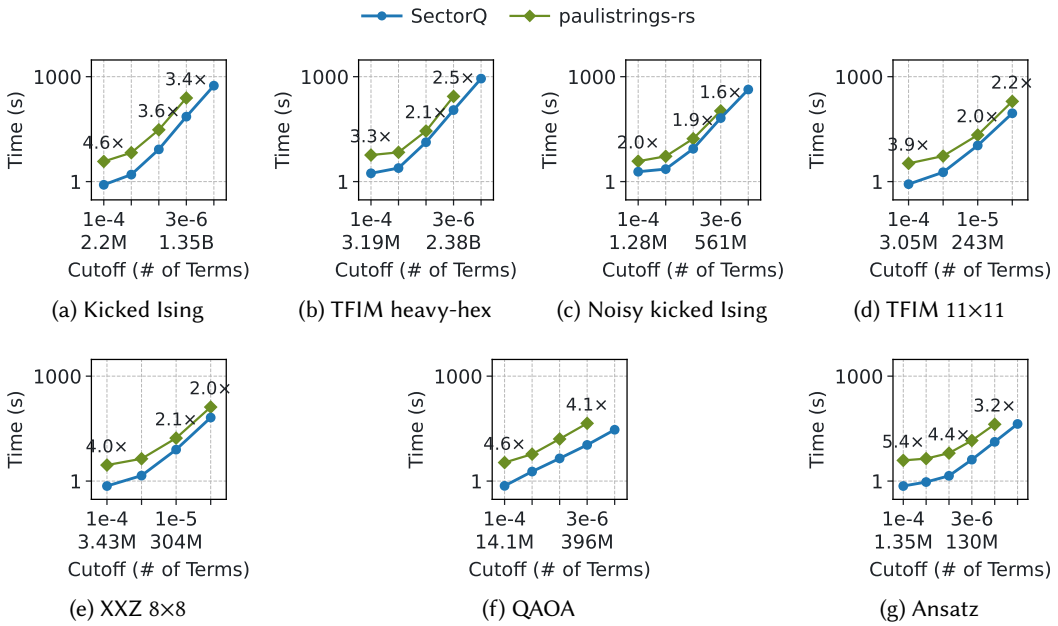



Fig. 11. *SectorQ* and fixed linear partitions on 16 GPUs

### 4.4 Comparison with Fixed Linear Partitions

This comparison separates an existing fixed-partition implementation from a controlled ablation within our engine.

**Same Truncation Rule.** Figure 11 compares *SectorQ* with paulistrings-rs on 16 GPUs using truncation after each operation to match its gate-level coefficient decisions. Both implementations receive the same original circuit, and the fixed partition uses the library's seeded rows. Across 75 completed settings on one, four, and 16 GPUs, the expectation values agree within $4.7 \times 10^{-11}$. On 16 GPUs, *SectorQ* is 1.6-5.4 times faster over all 30 shared settings, and 1.6-4.1 times faster at the largest shared setting of each circuit. At the next tested cutoff in five circuits, paulistrings-rs exhausts GPU memory while *SectorQ* completes in 29-880 s. Small-setting timings include any lazy GPU kernel compilation in the propagation call. The comparison uses the implementation's fixed partitions and distinguishes their time and capacity from the within-engine ablation below.

**Fixed versus Recomputed Rows.** Figure 12 isolates row recomputation within the same *SectorQ* engine, retaining its Clifford frame, GPU operations, and truncation schedule. The fixed variant selects rows at the first partitioned window and retains them for the run, ending a window when a rotation changes those rows. The recomputed variant selects rows for each window. On 16 GPUs, fixed rows are 2.4-17.0 times slower over the successful paired settings, with identical printed expectation values in all 25 pairs. On four GPUs, the range is 1.3-14.8 times over 22 pairs. The smallest difference occurs on a small heavy-hex TFIM setting. At lower cutoffs, several fixed-row runs exhaust host memory while recomputed rows complete. Noisy kicked Ising is excluded because the fixed variant does not support its explicit Clifford operations between branching rotations. Table 6 also shows fewer windows and less inter-node traffic at a common cutoff. These measurements support window-dependent partitions independently of differences between simulator implementations.

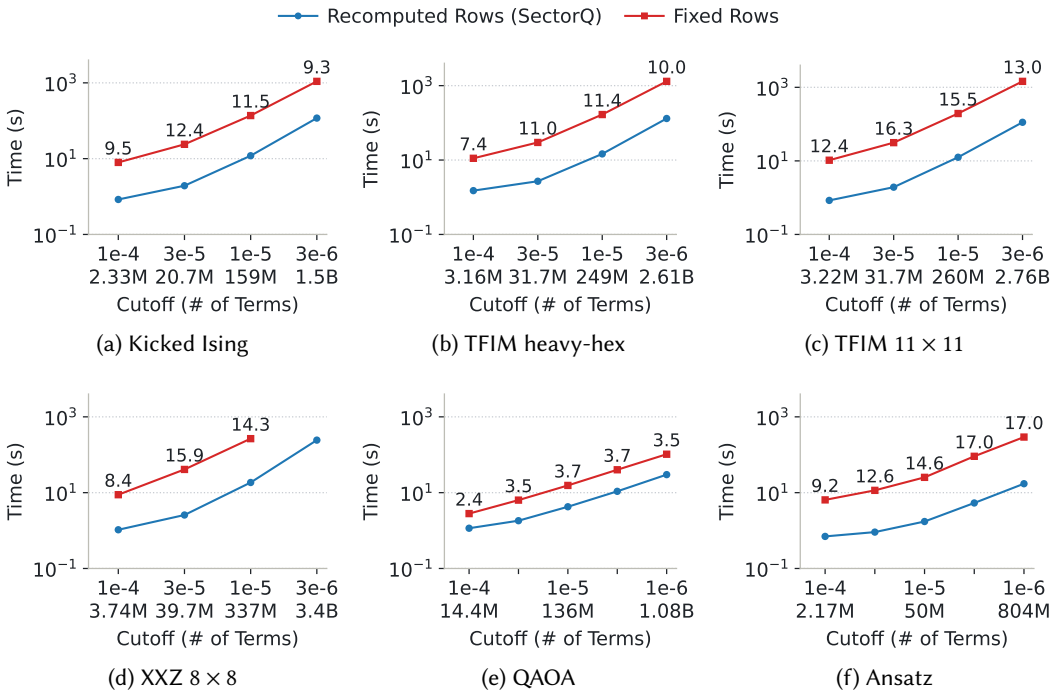



Fig. 12. Fixed and recomputed rows on 16 GPUs, with fixed-to-recomputed time ratios

Table 6. Windows and inter-node traffic at cutoff $10^{-5}$ on 16 GPUs

| | Windows | | Traffic (GB) | |
|---|---|---|---|---|
| **Circuit** | **Recomputed** | **Fixed** | **Recomputed** | **Fixed** |
| Kicked Ising | 20 | 878 | 45.2 | 698.5 |
| TFIM heavy-hex | 45 | 2,241 | 79.0 | 342.2 |
| TFIM 11 × 11 | 24 | 1,518 | 60.0 | 625.9 |
| XXZ 8 × 8 | 23 | 1,155 | 44.8 | 2,369.9 |
| QAOA | 8 | 198 | 8.2 | 34.0 |
| Ansatz | 34 | 1,392 | 8.3 | 107.7 |

### 4.5 Multi-Node Scalability

We examine increasing problem size and fixed problems as nodes are added, using original-circuit inputs for the primary baseline.

**Weak Scalability.** Figure 13 increases the retained term count with the number of GPUs by lowering the cutoff. Efficiency is the achieved boundary-term throughput per GPU relative to the corresponding one-GPU run, accounting for the term counts actually produced. For the large load, *SectorQ* retains 80-88% of that throughput on 16 GPUs and 72-80% on 64 GPUs across the three circuits. The range falls to 23-38% on 512 GPUs under the default sector cap. The measured *cuPauliProp* large-load configurations exhaust GPU memory, whereas the smaller load permits time comparisons on both systems. At 16 GPUs, *SectorQ* is 8.1-12 times faster than the original-circuit primary baseline on the small load. CPU results are shown where completed, with missing

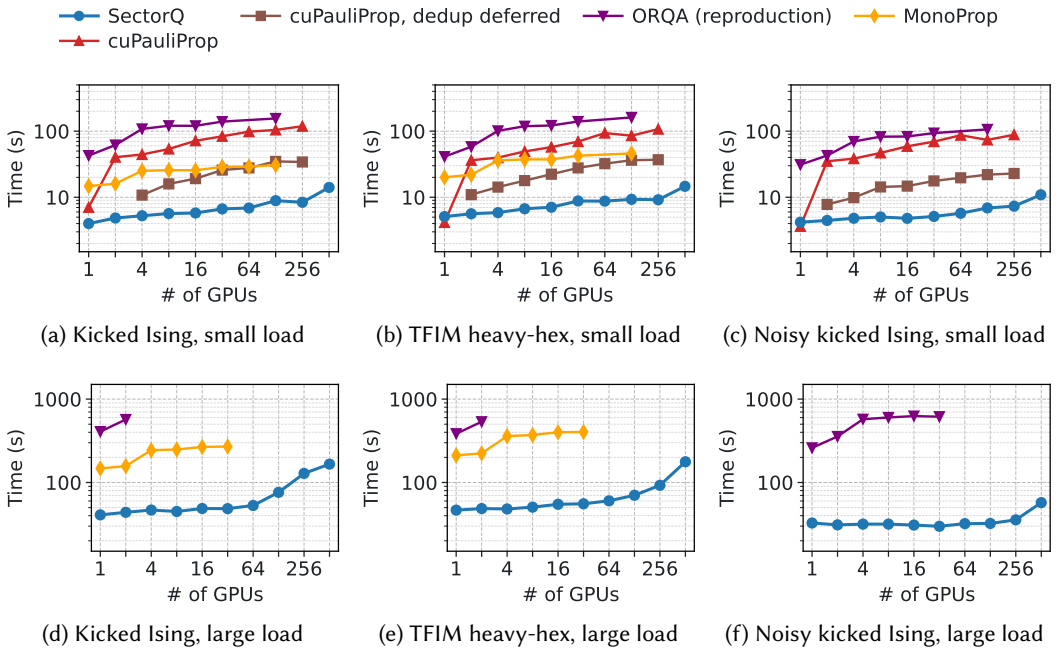



Fig. 13. Weak scaling from 1 to 512 GPUs

measurements omitted rather than interpreted as failures. The large-scale decomposition below distinguishes the cost of boundary exchange from insufficient concurrent sector tasks.

**Strong Scalability.** For fixed large problems, *SectorQ* achieves speedups of 81-166 on 256 GPUs and 83-180 on 512 GPUs relative to one GPU. The three representative settings are kicked Ising at cutoff $3 \times 10^{-6}$ and heavy-hex TFIM and noisy kicked Ising at $5 \times 10^{-6}$. Figure 14 additionally shows the kicked Ising cutoff ladder from one to 512 GPUs. The gain from 256 to 512 GPUs is modest under the default cap of 256 sectors. At kicked Ising cutoff $3 \times 10^{-6}$, time decreases from 10.48 to 9.67 s, while average worker idle time rises from 9% to 40% of wall time. The trace contains at most 256 nonempty sectors and no GPU splits in this run, limiting concurrent sector execution below the worker count. Small problems also saturate, with kicked Ising at cutoff $10^{-5}$ near three seconds from 128 GPUs onward. This behavior reflects task granularity and fixed costs as well as communication.

**Three Circuits on 128 GPUs.** The original-circuit baseline measurements on 128 GPUs are complete for the three representative sweeps. Figure 15 uses those results for kicked Ising and its noisy variant, while the TFIM input is already identical with and without a Clifford frame. At the largest measured setting completed by the primary baseline in each sweep, *SectorQ* is 7.4-13.4 times faster. For kicked Ising at cutoff $5 \times 10^{-6}$, the baseline takes 101.6 s and *SectorQ* takes about 7.6 s. The CPU comparisons use the same 32 nodes. MonoProp remains competitive at the smallest settings, while its different truncation rule prevents interpretation of these times as equal-error comparisons. The additional CPU cutoff ladders remain incomplete and their curves end at the last completed point. Thus, the 128-GPU comparison uses the same primary-baseline input policy as the 16-GPU results without inferring outcomes for pending runs.

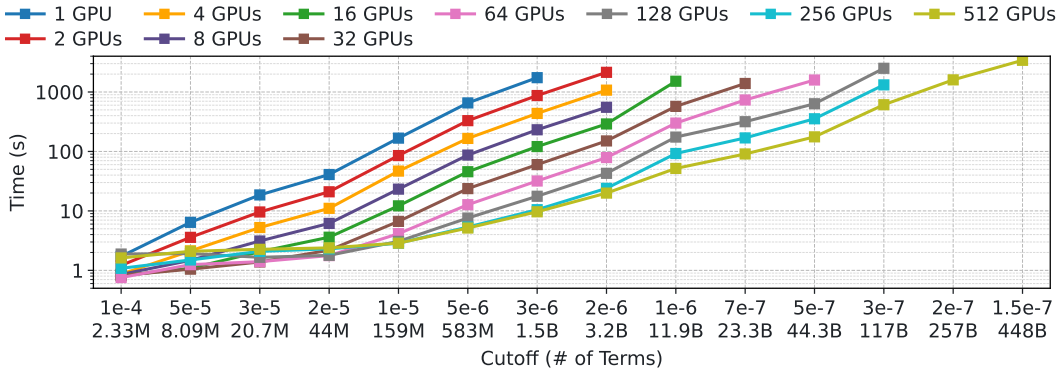
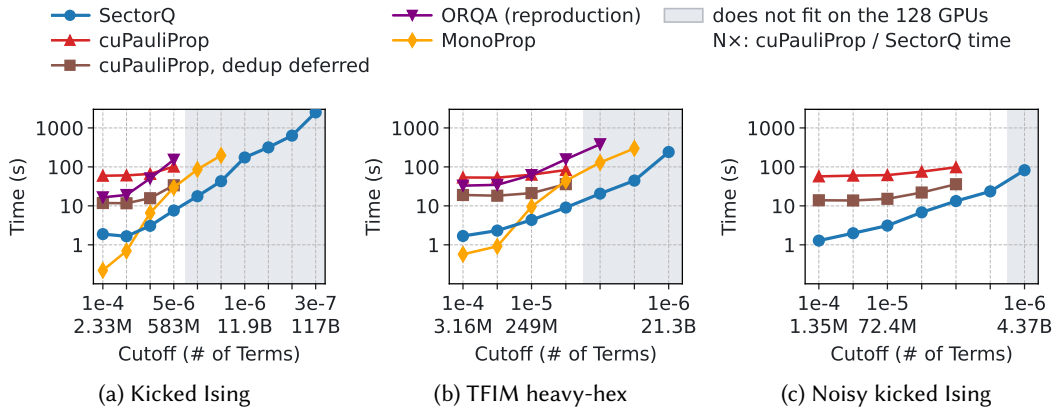



Fig. 14. Kicked Ising propagation from 1 to 512 GPUs




(a) Kicked Ising (b) TFIM heavy-hex (c) Noisy kicked Ising

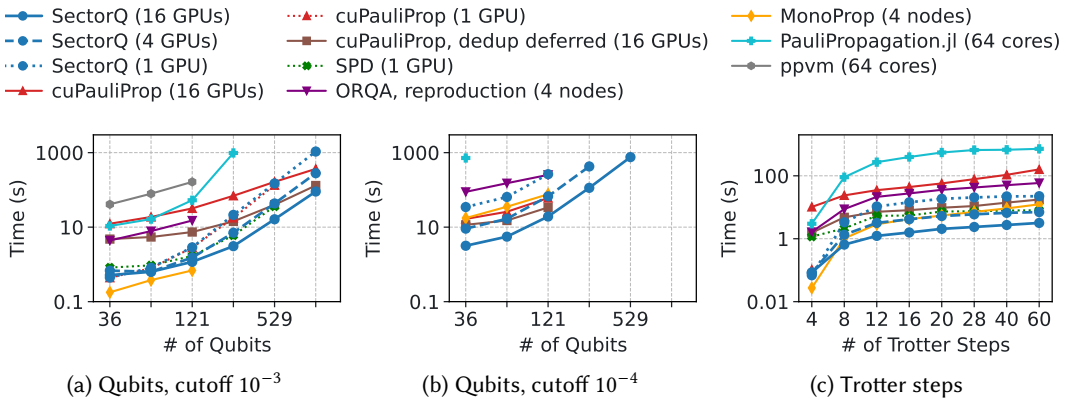
Fig. 15. Three circuits on 128 GPUs with original-circuit primary baselines

### 4.6 Circuit Size and Depth

Additional sweeps vary the circuit dimensions beyond the main seven-circuit suite.

**Number of Qubits.** Figure 16 varies a two-dimensional TFIM lattice from 36 to 1,024 qubits and propagates its total magnetization. This experiment changes the string length, graph size, and observable size together, rather than changing only the coefficient cutoff. At 1,024 qubits on a 32×32 lattice, *SectorQ* on 16 GPUs finishes in 90.4 s and is 4.1 times faster than the primary *cuPauliProp* baseline. The implementation extends the binary generator and row operations to the wider strings required by these lattices. The largest qubit count is the supported library limit in this experiment, not a measured limit on the mathematical partition rule. As in the main sweep, implementations that reject the qubit count are distinguished from executions that exceed memory or time. The result shows that the sector construction applies beyond the 127-qubit heavy-hex workload used for the largest capacity runs.

**Circuit Depth.** The depth experiment keeps the kicked Ising graph fixed and increases the number of Trotter steps. Longer propagation changes both the number of windows and the population of terms encountered within them. At 60 steps, *SectorQ* on 16 GPUs completes in 160 s, 50 times faster than the original-circuit *cuPauliProp* distributed baseline. The same window construction is applied throughout the deeper circuit without a partition chosen for the final operator in advance. The




(a) Qubits, cutoff $10^{-3}$ (b) Qubits, cutoff $10^{-4}$ (c) Trotter steps

Fig. 16. Simulation time by circuit size and depth

curve in Figure 16 therefore measures the combined cost of additional operations, changing term counts, and repeated boundary exchanges. A deeper circuit does not imply a proportional increase in retained terms, since truncation can remove branches introduced earlier. We consequently compare the measured times at each depth rather than extrapolate from the operation count alone. The result supports the execution mechanism over longer sequences of windows as well as larger graphs.

### 4.7 Extreme Scale-out

The capacity experiments lower the kicked Ising cutoff until execution or host storage limits the next step.

**Largest Finished Problems.** The kicked Ising capacity ladder reaches 448.2 billion boundary terms on 512 GPUs at cutoff $1.5 \times 10^{-7}$. This expansion occupies 17.9 TB at 40 bytes per term and completes in 3,413 s. The original-circuit *cuPauliProp* baseline on the same 512 GPUs finishes at cutoff $2 \times 10^{-6}$, corresponding to 3.2 billion boundary terms, and fails at the next tested cutoff. The completed problem is therefore 140 times larger by boundary term count. This ratio compares *SectorQ* specifically with *cuPauliProp*, not with all CPU implementations. On 128 GPUs, *SectorQ* completes 117.5 billion terms, while the 256-GPU endpoint remains under remeasurement after a stalled run. We retain its completed results without declaring the unresolved next step a memory limit. The capacity comparisons also distinguish measured failures from larger settings rejected because their retained terms alone exceed installed host memory.

**Host Memory Use.** The 512-GPU capacity run allocates 21,079 GiB in host chunks across 128 nodes, 64% of their installed host memory. This counter measures the chunk pool, not the entire resident memory of every process. The ratio to the boundary expansion includes reusable chunks and the transient storage needed for execution and exchange. The round-based transfer prevents the node from reserving a complete second receive expansion before it releases sent chunks. For the largest run, this representation supports a stored operator substantially larger than aggregate GPU memory while each worker processes a bounded sector. The next cutoff on the ladder is not reported as a completed or measured out-of-memory run when its footprint is only estimated. Host capacity remains a practical limit, and a single oversized sector must still admit a valid finer partition that fits the GPU working buffers.

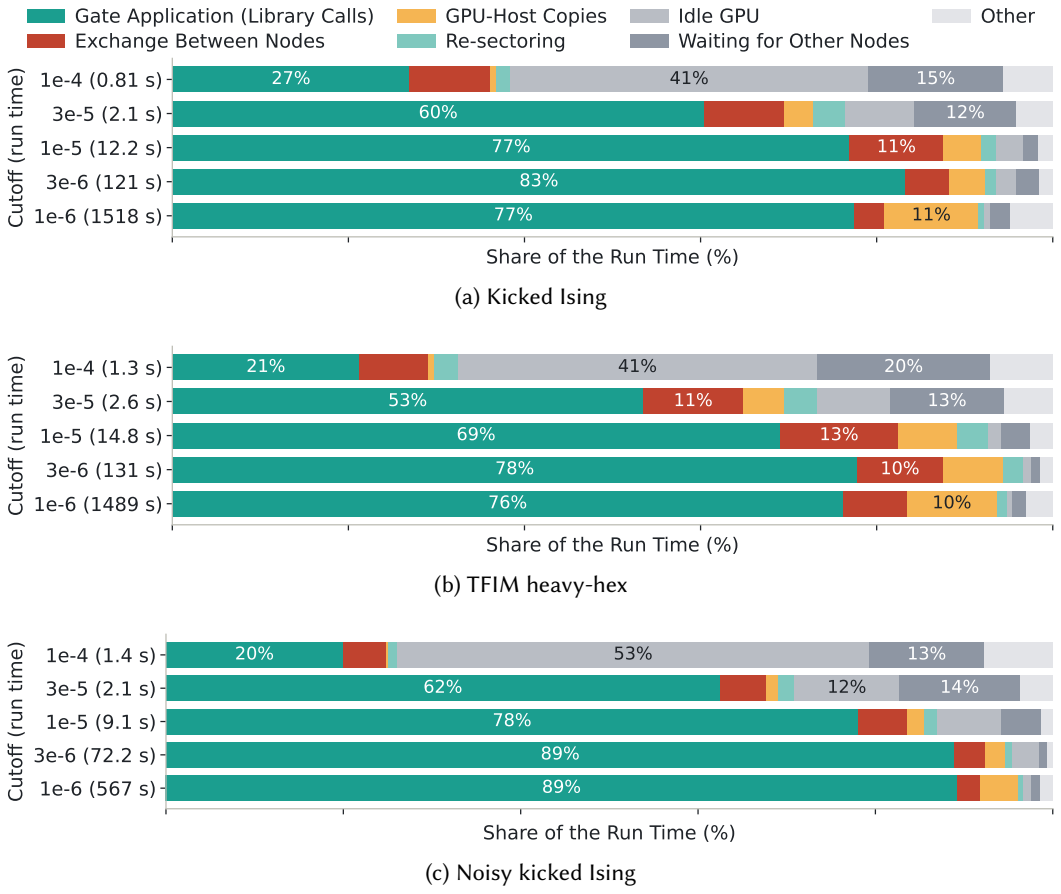



Fig. 17. Run time of *SectorQ* by activity on 16 GPUs

### 4.8 Time Analysis

The time counters identify the costs that remain after the distributed merge is confined to boundaries.

**Where the Time Goes.** Figure 17 divides the accumulated worker counters by the GPU count and compares them with wall time. The operation category measures time inside library calls, not sampled GPU utilization. For large 16-GPU settings, these calls account for 69-89% of the run, and inter-node exchange accounts for 3-13%. The separate MPI instrumentation of the original-circuit *cuPauliProp* baseline records 39-44% of its large-setting time inside MPI calls. Figure 18 extends the *SectorQ* decomposition to 128, 256, and 512 GPUs at cutoff $3 \times 10^{-6}$. Idle workers become a major component at 512 GPUs under the default sector cap. The largest 512-GPU capacity run uses a higher cap and has a different composition: library calls account for 46%, exchange for 26%, and GPU-host copies for 9%. Its exchange comprises 607 s in transfer and 280 s in posting requests, with count exchange below two seconds per node on average.

**Contribution of Each Element.** Figure 19 measures successive changes on kicked Ising at cutoff $10^{-5}$ and 16 GPUs. The original-circuit *cuPauliProp* baseline takes 95.0 s. A Clifford-frame input reduces this time to 54.3 s, and deferred global deduplication reduces it to 48.8 s. Window-sector

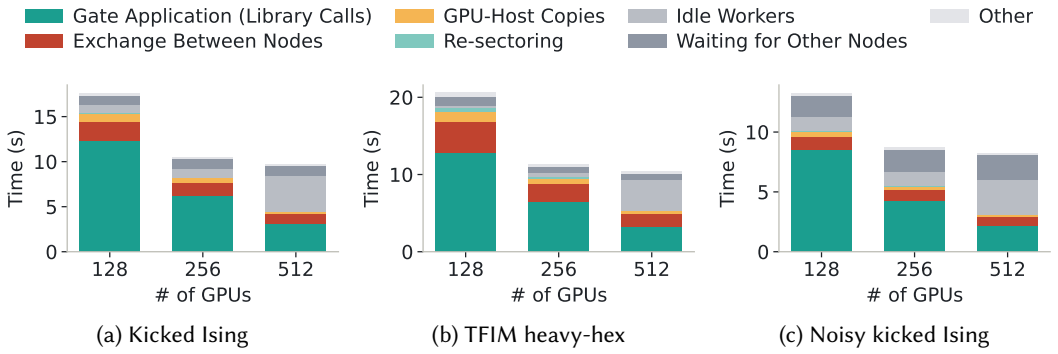



Fig. 18. Time by activity at cutoff $3 \times 10^{-6}$ on 128-512 GPUs

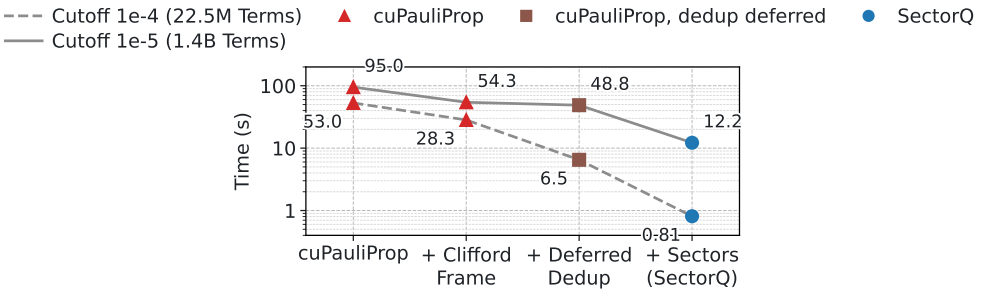


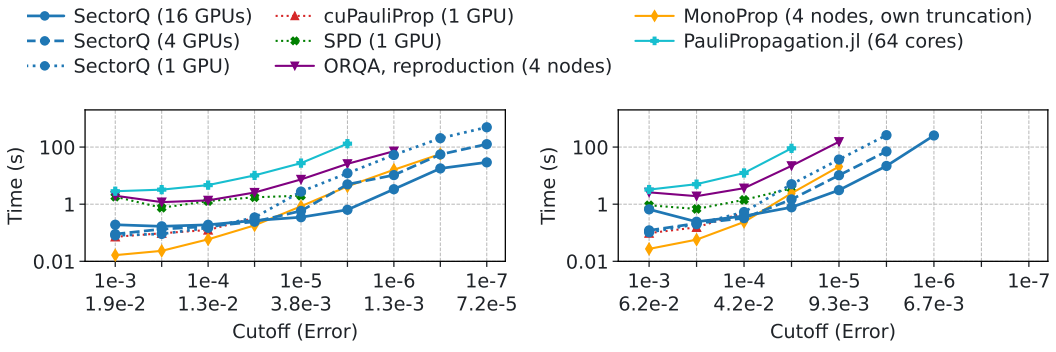

Fig. 19. Run time as each element is added on 16 GPUs

execution reaches 12.2 s with the same underlying library operations. The fixed-row ablation separately tests whether a partition chosen once can obtain the same result. At this setting, fixed rows take 137.1 s over 878 windows, whereas recomputed rows take 11.9 s over 20 windows. They send 698.5 and 45.2 GB between nodes, respectively. These runs come from a separate allocation from the element sequence, and their small timing difference is retained rather than merged into one measurement. Together, the two experiments attribute the benefit to both reduced global coordination and the particular choice of window-dependent sectors.

### 4.9 Accuracy

Numerical validation separates the distributed implementation from the approximation introduced by truncation.

**Exact Reference.** Figure 20 uses a 30-qubit TFIM on a $5 \times 6$ lattice with an exact state-vector reference. The reference calculation was checked against Pauli propagation without truncation on smaller grids. The four-step circuit reaches absolute error $7.2 \times 10^{-5}$ at cutoff $10^{-7}$, with *SectorQ* on 16 GPUs completing in 29.2 s. Methods with the same truncation rule produce the same error at a given cutoff, making their time comparison meaningful at that error. MonoProp has a separate error because its coefficient decisions differ. The six-step circuit requires substantially more work and reaches the time or memory limit at higher cutoffs. Neither sweep is strictly monotonic in error at every intermediate cutoff. The results therefore establish the cost of specific numerical approximations against a known answer, rather than a general error bound implied by the cutoff.




(a) 4 Trotter steps (b) 6 Trotter steps

Fig. 20. Simulation time and error of a 30-qubit TFIM

**Convergence of Kicked Ising.** The large kicked Ising workload lacks an exact reference at the evaluated scale. Its expectation values at successively lower cutoffs from $10^{-6}$ to $1.5 \times 10^{-7}$ are approximately 0.3217, 0.3256, 0.3265, 0.3226, 0.3153, and 0.3089. These values span about 0.018 and are not monotonic. A lower cutoff therefore cannot be presented as a measured guarantee of smaller physical error for this circuit. The scalability result is that *SectorQ* computes the same truncated propagation as the matched baselines where both finish and extends the computation to lower cutoffs with more retained terms. The distinction matters because the capacity improvement makes larger approximations accessible without proving their convergence. The exact-reference experiment and the cross-implementation agreement address complementary questions: approximation error in a tractable circuit and faithful distributed execution of a specified truncation rule.

### 4.10 Sensitivity to Parameters

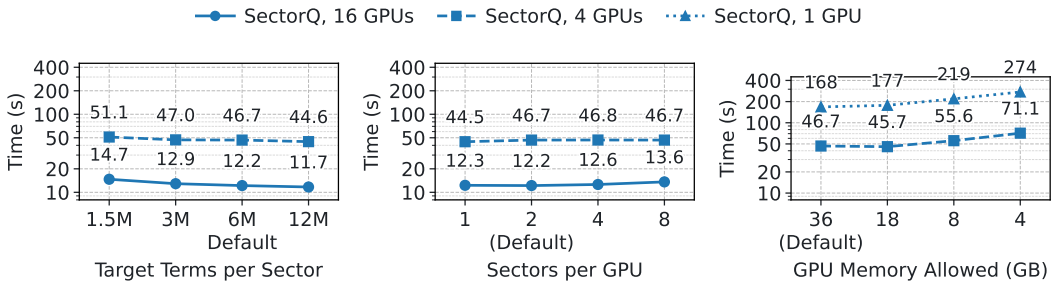
Figure 21 varies sector size, tasks per GPU, and the available GPU memory.

**Sector Size and Device Memory.** At cutoff $10^{-5}$, reducing the target sector size to one quarter of the default increases time by 9-26%, whereas doubling it reduces time by 2-9% across the representative circuits. More sectors increase the number of library calls, while larger sectors approach the buffer limit earlier during a window. One to four tasks per GPU differ by at most 9% in these measurements, and eight tasks can increase time by up to 35% on 16 GPUs. A separate device-memory experiment reduces the budget from 36 to 4 GB with unchanged expectation values. The smallest budget increases time by 12-64% but still completes the tested settings through finer sector execution. These results support the separation of stored capacity from per-GPU working memory. They do not imply that one sector cap is optimal at every cluster size, as the 512-GPU decomposition demonstrates. A separate 512-GPU kicked Ising run at cutoff $3 \times 10^{-6}$ with a cap of 4,096 sectors and smaller host chunks takes 11.6 s, compared with 9.7 s under the default configuration. This joint parameter change does not isolate the cap, but it shows that greater available concurrency alone need not improve the measured time.

### 4.11 Performance Variability

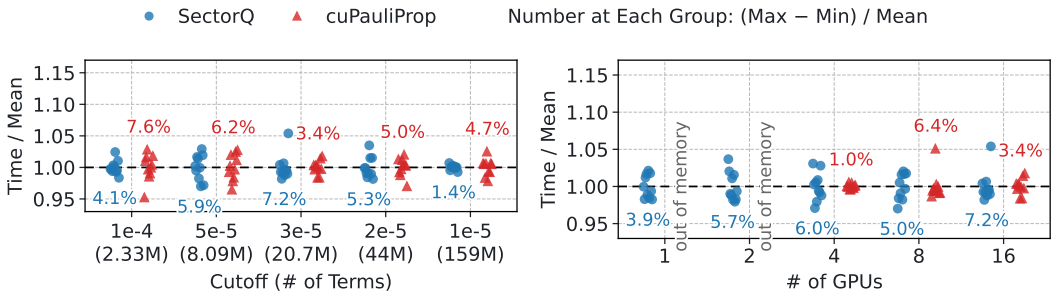
Figure 22 reports ten repetitions of representative kicked Ising settings.

**Repeated Runs.** The repetition experiment contains 160 completed runs over problem sizes and GPU counts. We define spread as the maximum minus minimum time divided by the mean of ten runs. On 16 GPUs, the spread is 1.4-7.2% for *SectorQ* and 3.4-7.6% for the original-circuit




(a) Target terms per sector (b) Sectors per GPU (c) GPU memory allowed

Fig. 21. Run time by parameter on kicked Ising at cutoff $10^{-5}$




(a) 16 GPUs by problem size (b) Cutoff $3{\times}10^{-5}$ by GPU count

Fig. 22. Run time of ten repeated runs

*cuPauliProp* baseline. All repeated runs preserve the printed expectation value. Each system's 16-GPU repetitions use one allocation, but the two systems use different allocations. This choice avoids mixing allocation changes into a system's repeated-run spread while retaining that distinction in the comparison. At cutoff $10^{-5}$, the ratio of the ten-run means is 7.6, close to the main sweep's single-run ratio of 7.8. The repetition results therefore support the reported time difference at this setting without treating a single measurement as a confidence interval.

## 5 Related Work

The relevant distinctions concern the partition rule, when it changes, and the memory that holds the propagated operator.

**Conserved Operator Subspaces.** Symmetries and invariant subspaces are established tools in quantum simulation. Bravyi et al. identify binary symmetries of fermionic Hamiltonians and remove qubits after selecting a symmetry sector [4]. West et al. describe a commutator graph whose connected components identify invariant operator subspaces under a set of generators [19]. These results establish the algebraic basis for independent evolution and preclude a novelty claim based on parity conservation alone. *SectorQ* uses binary rows as labels on Pauli strings, with the ordinary binary inner product expressing invariance under generator XOR. The labels are not symmetry eigenvalues of a quantum state, and their sectors need not be individual connected components of the commutator graph. The system derives sufficiently fine labels for a finite window and replaces them when the next window has different generators. This construction supplies an

executable partition for distributed merge and truncation without requiring a useful nontrivial partition conserved by the entire circuit.

**Fixed Linear Partitions.** The closest implementation-level comparison is paulistrings-rs, whose partition label is a binary matrix times the Pauli bit vector [12]. Its linear map identifies one communication partner for each nonlocal generator delta, and its local coset execution already exploits independent groups of strings. The documented partition rows remain fixed during a run, and automatic row selection that accounts for both balance and remote work is listed as an open problem. The parallel iQCC implementation also uses a bitmask to assign Pauli words to processes and computes the destination of a product through XOR [6]. A predictable destination can still be remote. *SectorQ* requires every chosen row to annihilate every generator in the current window, placing all resulting merge partners in one sector throughout that window. It then selects rows again for the next window and schedules sectors from host memory. Our same-engine fixed-row ablation isolates this temporal choice, while the paulistrings-rs comparison measures complete implementations with matched truncation.

**Distributed CPU Propagation.** ORQA demonstrates large-scale propagation on Fugaku with up to one trillion Pauli strings and $2^{17}$ processes [5]. Its distribution map is a structured enumeration of string bits designed to limit communication partners while maintaining balance. It is not a generic hash, and the sparsity of its communication is central to its scaling. The map still distributes updates as the propagation advances through generators, rather than deriving a partition conserved by a window. MonoProp provides distributed Pauli and Majorana propagation through a C++ implementation with Python interfaces [1]. These systems establish that large operators can reside in distributed CPU memory. *SectorQ* addresses the additional requirement of executing sectors through GPU kernels while the operator remains in host memory. Our evaluation labels the ORQA implementation as a reproduction, identifies the CPU resources used, and treats MonoProp's different truncation rule separately from comparisons with identical coefficient decisions.

**Propagation Libraries and GPU Kernels.** Sparse Pauli dynamics and PauliPropagation.jl develop efficient observable simulation and truncation methods [3, 17]. Quantinuum's SPD provides GPU execution, and Qiskit's Pauli propagation package supports Clifford operations, Pauli rotations, and noise channels [15, 16]. These libraries improve the cost and coverage of propagation within their supported execution environments. NVIDIA's *cuPauliProp* supplies GPU operations on sparse Pauli expansions and a distributed execution mode [13]. *SectorQ* uses its single-GPU operations for arithmetic and compares against its original-circuit distributed mode. Consequently, the contribution concerns the locality and storage of those operations, not a replacement arithmetic kernel. The measured element sequence separates Clifford preprocessing and deferred deduplication from window-sector execution. The single-GPU comparison also retains cases in which another implementation is faster. This distinction identifies the system benefit as the ability to execute growing operators across nodes without requiring the complete local expansion to remain in GPU memory.

**Repeated Evaluations and Gradients.** PADO-Pauli compiles circuits and observables into sparse matrices that are reused for batches of parameter values and supports gradients through PyTorch [9]. Its reuse of a compiled representation targets repeated evaluations during variational training. Backpropagating Pauli Propagation develops gradient evaluation whose computational complexity matches forward propagation and reduces its memory requirements through reversibility [11]. These objectives differ from the single forward propagation measured in our capacity and scaling experiments. *SectorQ* follows the surviving terms of one specified circuit and cutoff, with new rows chosen from those live terms at window boundaries. It neither assumes a fixed sparse program shared by parameter batches nor evaluates derivatives in the reported experiments. The distinction is relevant because a faster batched or gradient evaluation is not directly comparable to the time of

a single forward run with the same coefficient truncation. The methods address complementary workloads, and their integration would require a separate numerical and performance evaluation.

**Staged State Vector Simulation.** State vector simulators already reduce communication through groups of gates. Häner and Steiger schedule gates to reduce communication in a distributed simulation of 45 qubits [8]. Atlas partitions circuits into stages with an integer program, selects kernels with dynamic programming, and supports host-memory offloading [20]. Stage boundaries and host storage are therefore not new independently. Their state vector has a fixed index space, whereas Pauli propagation creates strings, combines equal strings, and removes coefficients during execution. A useful partition must preserve these merges before truncation, and the sizes of its parts depend on the surviving operator. *SectorQ* derives this locality from the window generators and obtains execution granularity from a sample of the live terms. An oversized sector can be subdivided only by rows valid for its remaining operations. This constraint connects host-memory execution to the algebraic partition and distinguishes the proposed mechanism from applying a state-vector stage schedule to a sparse operator.

## 6 Conclusion

In this paper, we design and implement *SectorQ*, a Pauli propagation system for GPU clusters that partitions the terms by the parities that each window of gates conserves. By recomputing these parities for every window from the generators of its gates and the live terms, we confine the merging of terms to one sector during a window and hold the sectors in host memory. We evaluate *SectorQ* with seven circuits on up to 512 NVIDIA A100 GPUs, reducing the simulation time of *cuPauliProp* by 4.4-9.7× on 16 GPUs and increasing the largest finished problem by 140× on 512 GPUs through exchanges at window boundaries and sectors in host memory.

## References


[1] Algorithmiq. 2026. MonoProp: Classical Simulation through Pauli and Majorana Propagation. https://github.com/Algorithmiq/monoprop Software repository.

[2] Frank Arute et al. 2019. Quantum supremacy using a programmable superconducting processor. *Nature* 574 (2019), 505–510. doi:10.1038/s41586-019-1666-5

[3] Tomislav Begušić, Johnnie Gray, and Garnet Kin-Lic Chan. 2024. Fast and converged classical simulations of evidence for the utility of quantum computing before fault tolerance. *Science Advances* 10 (2024), eadk4321. doi:10.1126/sciadv.adk4321

[4] Sergey Bravyi, Jay M. Gambetta, Antonio Mezzacapo, and Kristan Temme. 2017. Tapering off qubits to simulate fermionic Hamiltonians. arXiv:1701.08213 https://arxiv.org/abs/1701.08213

[5] Lukas Broers, Rong-Yang Sun, and Seiji Yunoki. 2026. Scalable Simulation of Quantum Many-Body Dynamics with Or-Represented Quantum Algebra. *Physical Review Applied* 26 (2026), 024046. doi:10.1103/y8ft-m61w

[6] Scott N. Genin, Ohyun Kwon, Seyyed Mehdi Hosseini Jenab, Seon-Jeong Lim, Taehyung Kim, Tae-Gon Kim, Rami Gherib, Angela F. Harper, Ilya G. Ryabinkin, and Michael G. Helander. 2025. Towards Quantum Advantage in Chemistry. arXiv:2512.13657 https://arxiv.org/abs/2512.13657

[7] I. M. Georgescu, S. Ashhab, and Franco Nori. 2014. Quantum simulation. *Reviews of Modern Physics* 86 (2014), 153–185. doi:10.1103/RevModPhys.86.153

[8] Thomas Häner and Damian S. Steiger. 2017. 0.5 Petabyte Simulation of a 45-Qubit Quantum Circuit. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. Association for Computing Machinery, Article 33, 10 pages. doi:10.1145/3126908.3126947

[9] Hyunwoo Kim and Youngseok Lee. 2026. PADO-Pauli: A GPU-Accelerated Implementation of Pauli Propagation. arXiv:2610.17755 https://arxiv.org/abs/2610.17755

[10] Youngseok Kim, Andrew Eddins, Sajant Anand, Ken Xuan Wei, Ewout van den Berg, Sami Rosenblatt, Hasan Nayfeh, Yantao Wu, Michael Zaletel, Kristan Temme, and Abhinav Kandala. 2023. Evidence for the utility of quantum computing before fault tolerance. *Nature* 618 (2023), 500–505. doi:10.1038/s41586-023-06096-3

[11] Sheng-Hsuan Lin, Etienne Granet, Kevin Hémery, and Henrik Dreyer. 2026. Backpropagating Pauli Propagation. arXiv:2607.15184 https://arxiv.org/abs/2607.15184

[12] lkdvos. 2026. paulistrings-rs. https://github.com/lkdvos/paulistrings-rs/tree/99c34367f5cae3bb1f250fd4b1a56a8285f77e63 Software repository, commit 99c3436.

[13] NVIDIA. 2026. cuPauliProp: A High-Performance Library for Pauli Propagation Quantum Simulators. https://docs.nvidia.com/cuda/cuquantum/latest/cupauliprop/index.html Version 0.5.0 used in the evaluation.

[14] John Preskill. 2018. Quantum Computing in the NISQ era and beyond. *Quantum* 2 (2018), 79. doi:10.22331/q-2018-08-06-79

[15] Qiskit contributors. 2026. Pauli Propagation. https://github.com/Qiskit/pauli-prop Software repository.

[16] Quantinuum. 2026. SPD: Sparse Pauli Dynamics. https://github.com/Quantinuum/spd Software repository.

[17] Manuel S. Rudolph, Tyson Jones, Yanting Teng, Armando Angrisani, and Zoë Holmes. 2026. Pauli Propagation: A Computational Framework for Simulating Quantum Systems. *PRX Quantum* 7 (2026), 032001. doi:10.1103/6vd7-l9bn

[18] Joseph Tindall, Matthew Fishman, E. Miles Stoudenmire, and Dries Sels. 2024. Efficient Tensor Network Simulation of IBM's Eagle Kicked Ising Experiment. *PRX Quantum* 5 (2024), 010308. doi:10.1103/PRXQuantum.5.010308

[19] Maxwell West, Neil Dowling, Angus Southwell, Martin Sevior, Muhammad Usman, Kavan Modi, and Thomas Quella. 2025. A graph-theoretic approach to chaos and complexity in quantum systems. *SciPost Physics Core* 8 (2025), 081. doi:10.21468/SciPostPhysCore.8.4.081

[20] Mingkuan Xu, Shiyi Cao, Xupeng Miao, Umut A. Acar, and Zhihao Jia. 2024. Atlas: Hierarchical Partitioning for Quantum Circuit Simulation on GPUs. In *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 17 pages. https://www.pdl.cmu.edu/ftp/associated/Xu-AtlasSC24.pdf